Water App

A full-stack web application for managing water service bookings. Built with React frontend and Node.js/Express backend.

🚀 Features

- **Service Booking System**: Customers can book water services
- **Customer Management**: Store customer information (name, address, phone)
- **Service Scheduling**: Date-based appointment scheduling
- **Message System**: Optional customer messages for special requests
- **RESTful API**: Clean API endpoints for data management
- **Session Management**: Secure user sessions with MongoDB storage

🛠️ Tech Stack

Frontend

- **React** - UI framework
- **Vite** - Build tool and dev server
- **CSS** - Styling

Backend

- **Node.js** - Runtime environment
- **Express.js** - Web framework
- **MongoDB** - Database
- **Mongoose** - ODM for MongoDB
- **Passport.js** - Authentication middleware
- **Express Session** - Session management
- **Connect-Mongo** - MongoDB session store

📋 Prerequisites

Before running this application, make sure you have:

- **Node.js** (v16 or higher)
- **npm** or **yarn**

- **MongoDB** (local installation or MongoDB Atlas)
- **Git**

🔧 Installation & Setup

1. Clone the Repository

```bash
git clone <your-repository-url>
cd waterApp
```

2. Backend Setup

```bash
# Navigate to server directory
cd server

# Install dependencies
npm install

# Create environment file
cp .env.example .env
# Edit .env with your configuration
```

3. Frontend Setup

```bash
# Navigate to client directory
cd ../client

# Install dependencies
npm install
```

4. Environment Configuration

Create a `.env` file in the `server` directory:

```env
```

```
# Database
MONGODB_URI=mongodb://localhost:27017/waterapp

# Server
PORT=5000

# Session
SESSION_SECRET=your-super-secret-session-key-here

# Environment
NODE_ENV=development
```

🚀 Running the Application

Development Mode

You need to run both frontend and backend servers:

Terminal 1 - Backend Server

bash

```
cd server
npm run dev
# or
npm start
```

Terminal 2 - Frontend Server

bash

```
cd client
npm run dev
```

Access the Application

- **Frontend**: http://localhost:3000 (or Vite's assigned port)
- **Backend API**: http://localhost:5000
- **API Health Check**: http://localhost:5000/api/health

📁 Project Structure

```
waterApp/
├── client/                 # React frontend
│   ├── public/
│   ├── src/
│   │   ├── components/     # Reusable components
│   │   ├── pages/          # Page components
│   │   ├── App.jsx         # Main app component
│   │   ├── main.jsx        # Entry point
│   │   ├── App.css         # Global styles
│   │   └── index.css       # Base styles
│   ├── package.json        # Frontend dependencies
│   ├── vite.config.js      # Vite configuration
│   └── index.html          # HTML template
├── server/                 # Node.js backend
│   ├── models/             # Database models
│   │   └── Booking.js      # Booking schema
│   ├── routes/             # API routes
│   ├── middleware/         # Custom middleware
│   ├── config/             # Configuration files
│   ├── server.js           # Main server file
│   └── package.json        # Backend dependencies
├── .gitignore              # Git ignore rules
└── README.md               # This file
```

🔗 API Endpoints

Bookings

- `GET /api/bookings` - Get all bookings
- `POST /api/bookings` - Create new booking
- `GET /api/bookings/:id` - Get specific booking
- `PUT /api/bookings/:id` - Update booking
- `DELETE /api/bookings/:id` - Delete booking

Health Check

- `GET /api/health` - Server health status
- `GET /` - API information

Request/Response Examples

Create Booking

bash

```
POST /api/bookings
Content-Type: application/json

{
  "name": "John Doe",
  "address": "123 Main St, City, State",
  "phoneNumber": "555-123-4567",
  "date": "2024-12-25T10:00:00Z",
  "service": "Water Tank Cleaning",
  "message": "Please call before arriving"
}
```

Response

json

```
{
  "_id": "64a1b2c3d4e5f6789",
  "name": "John Doe",
  "address": "123 Main St, City, State",
  "phoneNumber": "555-123-4567",
  "date": "2024-12-25T10:00:00Z",
  "service": "Water Tank Cleaning",
  "message": "Please call before arriving",
  "createdAt": "2024-07-20T10:30:00Z",
  "updatedAt": "2024-07-20T10:30:00Z"
}
```

🔒 Database Schema

Booking Model

javascript

```
{
  name: String (required),
  address: String (required),
  phoneNumber: String (required),
  date: Date (required),
  service: String (required),
  message: String (optional)
}
```

🧪 Testing

Manual Testing

- Use Postman or similar tool to test API endpoints
- Access frontend at development URL
- Check MongoDB for data persistence

Future Testing Setup

```bash
# Backend tests (when implemented)
cd server
npm test

# Frontend tests (when implemented)
cd client
npm test
```

🚀 Deployment

Production Build

```bash
# Build frontend
cd client
npm run build

# The build files will be in client/dist/
```

Environment Variables for Production

```env
NODE_ENV=production
MONGODB_URI=your-production-mongodb-uri
SESSION_SECRET=your-production-secret-key
PORT=80
```

🤝 Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/AmazingFeature`)
3. Commit your changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request

📝 Scripts

Backend (server/)

```json
{
  "start": "node server.js",
  "dev": "nodemon server.js",
  "test": "jest"
}
```

Frontend (client/)

```json
{
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
}
```

🐛 Troubleshooting

Common Issues

"Cannot GET /" Error

- Make sure you have a root route defined in server.js
- Check that the server is running on the correct port

MongoDB Connection Issues

- Verify MongoDB is running locally or connection string is correct
- Check firewall settings for MongoDB Atlas

ES Module Errors

- Ensure models are in the server directory, not client
- Check package.json "type" field settings

Headers Already Sent Error

- Make sure each route sends only one response
- Add return statements after res.send()/res.json()

Logs

Check server logs for detailed error information:

```bash
cd server
npm start
```

📄 License

This project is licensed under the MIT License - see the LICENSE file for details.

👥 Team

- Developer: [Your Name]
- Contact: [Your Email]

🔮 Future Enhancements

- [ ] User authentication and authorization
- [ ] Email notifications for bookings
- [ ] Service provider dashboard
- [ ] Payment integration
- [ ] Mobile app development

- [ ] Advanced booking analytics
- [ ] SMS notifications
- [ ] Service rating system

Happy Coding! 💧